Contest Strategy, Debugging & Stress Testing

Turning knowledge into accepted solutions under pressure

Learning goals

- Plan the **first 30 minutes** of a contest and triage a problem set
- Map **constraints → target complexity** instantly
- Debug systematically: a **WA/RE/TLE checklist** instead of staring
- Use compiler **sanitizers** and debug-mode STL
- Build a **stress test** (generator + brute force + compare loop)
- Work as an ICPC **team of three with one computer**

Same knowledge, different results

Two contestants who know the same algorithms can finish far apart because of:

- **Order** in which problems are attempted
- **Time lost** on bugs that a checklist would catch in 2 minutes
- **Penalty** from careless wrong submissions
- **Getting stuck** — not switching problems when they should

Strategy is a skill — practice it in every virtual contest.

Scoring recap

Format	Rank by	Implication
ICPC	solved, then penalty = Σ (accept time + 20·wrong tries)	solve easy ones **fast**; don't submit blind
Codeforces	points decaying with time, −50 per wrong try	early solves are worth more
IOI / olympiad	partial points per subtask	always submit subtask solutions
AtCoder	points, time of last AC + 5 min per WA	similar to ICPC

The first minutes (ICPC)

1. **Split** the problem set: each member reads ~⅓ (A–D, E–H, I–L)
2. While reading: write a one-line summary + guessed difficulty per problem
3. **Fastest typist** starts coding the easiest problem immediately
4. Watch the **scoreboard**: problems solved by many teams early are easy
5. After ~20–30 min: everyone has read everything relevant; assign problems

Goal: first AC within ~10–15 minutes.

Triage

For each problem write:

```
C  | grid BFS, n,m ≤ 1000      | easy   | ~15 min | Ana
F  | DP? n ≤ 5000               | medium | ~40 min | Ben
H  | geometry, many cases       | hard   | ?       | later
```

- Sort by (estimated time / confidence), not by letter
- Revisit estimates as the scoreboard evolves
- A problem nobody solved after 2 hours is probably **hard**, not a trick

Constraints → complexity

n	Feasible complexity	Typical idea
≤ 10–12	$O(n!)$, $O(n \cdot n!)$	permutations, backtracking
≤ 20–25	$O(2^n \cdot n)$	bitmask, meet in the middle
≤ 500	$O(n^3)$	Floyd-Warshall, interval DP
≤ 5000	$O(n^2)$	2D DP
≤ $2 \cdot 10^5$	$O(n \log n)$, $O(n\sqrt{n})$	sorting, segment tree, Mo
≤ 10^7	$O(n)$	linear scans, sieve
≤ 10^{18}	$O(\log n)$, $O(\sqrt{n})$	math, binary search, matrix power

~10^8 simple operations per second is a safe estimate.

Before you type

- Re-read the **output format** and **constraints** (multi-test? sum of n bounded?)
- Solve the **samples by hand** with your idea
- Try a **small tricky case**: n = 1, all equal, all negative, maximum values
- Check value ranges: is `long long` needed? ($10^5 \times 10^5 = 10^{10} > 2^{31}$)
- Write the plan on paper (in ICPC: **while someone else uses the computer**)

Thinking at the keyboard wastes the team's only computer.

Implementation hygiene

- One tested **template** (fast I/O, `using ll = long long`, a debug macro)
- Clear names over clever macros; short functions
- `long long` by default when sums or products appear
- Reset **all** global state between test cases
- Use `vector` sized from input, not fixed arrays sized from guesses
- Avoid copy-pasting near-identical blocks — bugs hide in the copies

WA checklist

1. Overflow? (`int` products, sums, `1 << 40`)
2. Off-by-one? (`<` vs `<=`, 0/1-based, inclusive ranges)
3. Multi-test: arrays/globals **reset**? early `return` / `break` leaving input unread?
4. Edge cases: n = 1, empty answer, duplicates, negative numbers
5. Output format: spaces, newlines, "YES" vs "Yes", precision of doubles
6. Uninitialized variables, especially `ans` / `max` (should it start at −∞?)
7. Is the **algorithm** actually correct? (greedy without proof!)

Read your code **line by line** against the checklist — then stress test.

RE / TLE / MLE

Verdict	Usual suspects
RE	out-of-bounds index, deep recursion (stack), division by 0, `.back()` on empty
TLE	wrong complexity, `endl` flushing, `cin` without sync off, infinite loop, passing vectors by value
MLE	`int a[10^7][…]`, recursion memo too large, vector of vectors with high overhead

TLE can be a hidden WA (infinite loop) — and RE can be a hidden TLE (stack overflow from recursion).

Let the compiler help

```
g++ -std=c++17 -O2 -Wall -Wextra -Wshadow \
    -fsanitize=address,undefined -D_GLIBCXX_DEBUG sol.cpp -o sol
```

- `-Wall -Wextra -Wshadow`: unused/uninitialized/shadowed variables
- `-fsanitize=address`: out-of-bounds, use-after-free → exact line
- `-fsanitize=undefined`: signed overflow, bad shifts
- `-D_GLIBCXX_DEBUG`: bounds-checked `vector[]`, iterator checks

Use these **locally only**; submit with normal flags.

Debug output that never reaches the judge

```
#ifdef LOCAL
#define dbg(x) cerr << #x << " = " << (x) << endl
#else
#define dbg(x)
#endif
```

Compile locally with `-DLOCAL`.

- Print **intermediate states** (DP tables, BFS order), not just the answer
- Compare with values computed by hand on the sample
- `cerr` goes to stderr — judges ignore it, but it still costs time: remove heavy output

Stress testing

When you have a **WA** and no idea why:

```
gen.cpp    → random small test
brute.cpp  → obviously correct, slow solution
sol.cpp    → your fast solution
loop: generate → run both → compare → stop at first difference
```

- Small tests (n ≤ 6–10) make the failing case **readable**
- The brute force must be **simple enough to be right**
- Often finds the bug in < 1 minute

Generator

```cpp
#include <bits/stdc++.h>
using namespace std;
int main(int argc, char* argv[]) {
    mt19937 rng(atoi(argv[1]));                    // seed from the command line
    auto rnd = [&](int lo, int hi) {
        return uniform_int_distribution<int>(lo, hi)(rng);
    };
    int n = rnd(1, 6);
    cout << n << "\n";
    for (int i = 0; i < n; i++) cout << rnd(-10, 10) << " \n"[i == n - 1];
}
```

- Seed from the argument → every failing test is **reproducible**
- Small values make collisions/duplicates/negatives likely

The loop

```
#!/bin/bash
g++ -O2 -o gen gen.cpp && g++ -O2 -o brute brute.cpp && g++ -O2 -o sol sol.cpp
for i in $(seq 1 1000); do
    ./gen $i > in.txt
    ./sol < in.txt > out1.txt
    ./brute < in.txt > out2.txt
    if ! cmp -s out1.txt out2.txt; then
        echo "Mismatch on seed $i"; cat in.txt
        echo "sol:";   cat out1.txt
        echo "brute:"; cat out2.txt
        break
    fi
done
```

Multiple valid answers? Replace `cmp` with a **checker** program.

Example: a bug found in seconds

Maximum subarray sum, "fast" solution:

```
long long best = 0, cur = 0;
for (long long x : a) { cur = max(0LL, cur + x); best = max(best, cur); }
```

Stress test output (real run, n = 1):

```
Mismatch on seed 7
1
-6
sol:
0
brute:
-6
```

Bug: empty subarray allowed implicitly. Fix: `cur = max(x, cur + x)`, `best` starts at $-\infty$.

Assertions

```
assert(0 <= idx && idx < n);
assert(lo <= hi);                    // binary search invariant
assert(total == expected);           // e.g., flow conservation
```

- Turns a silent WA into an **RE** — which tells you where your belief is wrong
- Cheap insurance in contest code; remove only if they are slow

When you are stuck

- **20-minute rule:** no progress for ~20 min → explain the problem to a teammate or switch
- Re-read the statement — many "impossible" problems are misread
- Try small cases by hand / brute force and look for a **pattern**
- Change the angle: binary search the answer? think backwards? complement count? graph model?
- Sunk cost is real: an unsolved problem costs 0 penalty

ICPC: one computer, three people

Role	Does
Driver	types the current solution; keyboard time is precious
Thinker(s)	solve next problems **on paper**, prepare code outlines
Reviewer	reads printed code / partner-debugs WA submissions

- Debug **away** from the keyboard: print the code, let the next person type
- Before submitting: a second person checks the sample output and format
- Keep the team notebook (TRD) organized by topic

Team communication

- Short status updates: "C: idea confirmed, needs 15 min coding"
- **Never** two people silently working on the same problem by accident — keep a shared problem sheet
- Pair up on hard problems: one explains, one challenges
- Agree in advance who decides priorities in the last hour
- Stay calm: a wrong submission is information, not failure

The last hour

- Scoreboard **freezes**: you no longer see others' results
- Prioritize the problem **closest to AC**, not the most interesting
- Multiple people can work on different bugs of the same problem, printed
- Last 10–15 minutes: submit anything plausible (no penalty if it is never accepted in ICPC — penalty counts only for solved problems)

After the contest: upsolving

- The most efficient learning happens **after** the contest
- For every problem you **could almost** solve: read the editorial, implement it yourself
- Keep a **mistake log**: bug type, how to detect it next time
- Add new techniques to your **notebook** with tested code
- Weekly rhythm: 1 virtual contest + upsolving + topic practice

Practice problems

Drill the checklist on bug-prone classics:

- **CF 1A** Theatre Square (overflow) · **CSES 1068** Weird Algorithm (overflow)
- **CSES 1083** Missing Number (sum overflow) · **CSES 1643** Maximum Subarray Sum (all negative)
- **CF 466C** Number of Ways (off-by-one) · **CF 1352C** K-th Not Divisible by n (math edge cases)
- **CF 4C** Registration System (map, format)

Strategy practice: virtual **Codeforces Round 1520 (Div. 3)** and **AtCoder ABC 200** under real time limits; time every phase.

Summary

- Read everything early, triage by time/confidence, watch the scoreboard
- Translate constraints into a target complexity **before** coding
- WA → checklist → sanitizers → **stress test**
- Assertions turn wrong beliefs into visible crashes
- ICPC: the computer is the bottleneck — think and debug on paper
- Upsolve and log your mistakes

Next week: final review & mock contest.